Pytest

1. 未带参数装饰器

装饰器符合开闭原则：对扩展开放（主要实现功能的函数），对修改（对主要实现功能的函数添加额外行为）封闭。装饰器就是对被装饰函数添加一些额外行为，这些行为很常用，因此被写成一个装饰函数便于其他函数使用。

```
import functools
from datetime import datetime

'''
未带参数的装饰函数结构：

step1:
def screenshot_on_failure(func_main)

    return wraper

strep2:
    @functools.wraps(func_main) # 实现主要功能函数
    def wrapper(*args,**kargs):  # 是实现主要功能函数的参数
        try:
            return func_main(*args,**kargs) # 返回实现主要功能函数
        except Exception as e:
            raise e
step3: step2在step1里面

def screenshot_on_failuer(func_main):
    @functools.wraps(func_main)
    def wrapper(*args,**kargs):
        try:
            return func_main(*args,**kargs) # 实现主要功能的函数不能变，因此直接返回
        except Exception as e:
            # 此处可以添加一些操作
            self=args[0]    # args[0] 第一个参数是实例本身
            timestamp=datetime.now().strftime('%Y-%m-%d_%H:%M:%S')
            file_name=f'error_{func_main.__name__}_{timestamp}.png'
            self.driver.save_screenshot(file_name)
            print(f"❌ 操作失败，已截图保存: {filename}")
            raise e # raise 返回上层防止错误被吃掉
    return wrapper  # 此处返回的名字必须是同名
'''
```

例子未带参数截图：

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import functools
from datetime import datetime

def screenshot_on_failure(func):
```

```
    """失败自动截图装饰器"""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        try:
            return func(*args, **kwargs)
        except Exception as e:
            # 获取 self（实例对象）
            self = args[0]

            # 生成截图文件名：error_方法名_时间戳.png
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")  # %f是微秒，
避免重名
            filename = f"error_screenshot/error_{func.__name__}_{timestamp}.png"

            # 确保目录存在
            import os
            os.makedirs("error_screenshot", exist_ok=True)

            # 截图
            self.driver.save_screenshot(filename)
            print(f"[截图保存] 操作 {func.__name__} 失败，截图: {filename}")

            # 重新抛出异常（关键！不要吞异常）
            raise e
    return wrapper


class BasePage:
    def __init__(self, driver, timeout=10):
        self.driver = driver
        self.wait = WebDriverWait(driver, timeout)

    @screenshot_on_failure  # ← 加上装饰器！
    def find_element(self, locator):
        """查找元素（失败时自动截图）"""
        return self.wait.until(EC.visibility_of_element_located(locator))

    @screenshot_on_failure  # ← 加上装饰器！
    def click_element(self, locator):
        """点击元素（失败时自动截图）"""
        element = self.wait.until(EC.element_to_be_clickable(locator))
        element.click()

    @screenshot_on_failure  # ← 加上装饰器！
    def input_text(self, locator, text):
        """输入文本（失败时自动截图）"""
        element = self.find_element(locator)
        element.clear()
        element.send_keys(text)


class SearchPage(BasePage):
    URL = 'https://www.baidu.com'

    def __init__(self, driver):
        super().__init__(driver)
        self.search_input = (By.ID, "kw")
        self.search_button = (By.ID, "su")
```

```
    # 这个方法也继承了装饰器（因为调用了父类的 input_text）
    def search(self, keyword):
        self.input_text(self.search_input, keyword)
        self.click_element(self.search_button)
```

2. 带参数装饰器

带参数的装饰器，其实就是比未带参数的装饰器多一层函数，最外层是装饰器参数，第二层参数是函数作为参数，第三层是实现主要功能函数的参数。

```
'''
直接先写框架
step1(带参数):
def screenshot_on_failur(folder='error_screenshot',enabled=True)    # 使用关键字参数带有默认值

    return decorate
step2(装饰器):
    def decorate(func_main):    # 不带参数此处就直接起名：见名知义;由于带参数，所以此处的名字需要与step1的返回变量名一致

        reurn wrapper
step3(包装):
        @functools.wraps(func_main)
        def wrapper(*args,**largs): # 此处的函数名需要与step2的返回变量名同名
            try:
                return func_main(*args,**kargs) # 返回主要功能函数
            except Exception as e:

                raise e
'''
```

自定义目录或者是否截图：

```
import functools,datetime,os

def screenshot_on_failure(folder="error_screenshot", enabled=True): # setp1:带参数
    """带参数的装饰器工厂"""
    def decorator(func):    # step2:装饰器
        @functools.wraps(func)
        def wrapper(*args, **kwargs):   # step3:包装
            if not enabled:  # 可以关闭截图功能
                return func(*args, **kwargs)

            try:
                return func(*args, **kwargs)
            except Exception as e:
                self = args[0]
                timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
                filename = f"{folder}/error_{func.__name__}_{timestamp}.png"

                import os
                os.makedirs(folder, exist_ok=True)
                self.driver.save_screenshot(filename)
```

```
                print(f"[自动截图] {filename}")
                raise e
        return wrapper
    return decorator


class BasePage:
    @screenshot_on_failure(folder="screenshots/login", enabled=True)
    def click_login(self):
        pass

    @screenshot_on_failure(enabled=False)  # 这个操作不截图
    def get_title(self):
        return self.driver.title
```

3. 创建时序

我明白了，同级别，先写的先执行（按顺序），类中定义的变量分为实例变量，和类变量（类内部的块级别），实例变量需要实例化才会执行，类变量是直接定义（可供内外执行），装饰就像调用，遇见就即可执行。
至于if __name__=='__main__'只有单独运行该文件才会执行。

```
模块级代码 → 遇到类/函数定义 → 类/函数内部立即执行（装饰器、类变量）
→ 定义完成后继续模块级代码 → 最后才到 if __name__ == '__main__'
```

```
# ==========================================
# 文件: execution_order_demo.py
# 运行方式: python execution_order_demo.py
# ==========================================

print("【第一步-1】模块级代码开始执行 - 文件被读取")

# 模块级变量定义（第一步）
module_var = "我在模块级别定义"
print(f"【第一步-2】定义模块变量: {module_var}")

# 模块级函数定义（第一步 - 只定义不执行函数体）
def module_function():
    print("    我是函数内部，只有被调用时才执行（现在没调用）")
print("【第一步-3】定义了函数 module_function（但函数体未执行）")

print("\n===== 即将进入类定义（类装饰器会立即执行！） =====\n")

# ==========================================
# 【第一步-4】Python 看到 class 关键字，开始"建造"类
# ==========================================
class MyClass(object):
    # 【第二步】类体内部代码在类定义时立即执行！
    print("【第二步-1】类体内部代码开始执行（类定义时）")

    # 类属性（类定义时赋值）
    class_attr = "类属性"
    print(f"【第二步-2】定义类属性: {class_attr}")
```

```python
    # 【第二步-3】类装饰器/装饰器表达式立即执行！
    # 注意：这里模拟装饰器行为
    print("【第二步-3】类装饰器执行（如果有 @decorator）")

    def __init__(self):
        # 注意：__init__ 现在只是定义，不会执行！
        print("    我是 __init__，实例化时才执行（现在只是定义）")

    def method(self):
        # 同上，只是定义
        print("    我是普通方法，被调用时才执行")

    print("【第二步-4】类体内部代码执行完毕，即将生成类对象")

# 【第三步】类定义完成，MyClass 变量指向类对象
print(f"\n【第三步】类定义完成！MyClass = {MyClass}")
print(f"【第三步】类对象已创建，内存地址：{id(MyClass)}")

print("\n===== 类定义结束，回到模块级代码 =====\n")

# 模块级代码继续（还是第一步的范畴，因为模块还没导入完）
another_var = "另一个模块变量"
print(f"【第一步-5】类定义后，继续执行模块级代码：{another_var}")

# 创建实例（这依然是模块级代码，第一步的一部分）
print("\n【第一步-6】模块级代码：准备实例化 MyClass")
instance = MyClass()  # 此时才调用 __init__
print(f"【第一步-7】实例创建完成：{instance}")

print("\n========================================")
print("模块级代码执行完毕（文件已加载完成）")
print("========================================\n")

# ==========================================
# 【第四步】只有当直接运行脚本时，才执行这个块
# ==========================================
if __name__ == '__main__':
    print("【第四步-1】if __name__ == '__main__' 开始执行")
    print(f"【第四步-2】模块变量可用：{module_var}")
    print(f"【第四步-3】类可用：{MyClass}")

    # 可以再次实例化
    instance2 = MyClass()
    print(f"【第四步-4】在 main 中创建了新实例：{instance2}")

    print("\n【第四步-5】脚本执行完毕")
# 将以上代码转换为python格式
# 以下变为注释
# 直接运行（python demo.py）：
# 执行全部 4 个步骤
# 作为模块导入（import demo）：
# 只执行前 3 步
# 跳过第四步（if __name__ == '__main__' 不会执行）
# 这就是为什么你的 file_path 如果定义在 if __main__ 里，类装饰器（第二步）找不到它——类装饰器比 main 块先执行！
```

4. 实例方法，静态方法，类方法

方法类型	装饰器	第一个参数	调用方式	使用场景
实例方法	无	`self` (实例)	`obj.method()`	需要访问实例属性
静态方法	`@staticmethod`	无	`Class.method()` 或 `obj.method()`	逻辑上属于类，但不需要实例状态
类方法	`@classmethod`	`cls` (类)	`Class.method()` 或 `obj.method()`	需要访问类属性/方法

```python
import pytest
from selenium import webdriver
from typing import List, Dict

# ========== 1. 实例方法（Instance Method）==========
class TestLogin:
    def __init__(self):
        # 实例属性：每个实例独立的浏览器驱动
        self.driver = None
        self.base_url = "https://example.com"

    def setup_driver(self, browser="chrome"):
        """实例方法：操作实例属性 self.driver"""
        if browser == "chrome":
            self.driver = webdriver.Chrome()
        self.driver.get(self.base_url)
        return self.driver

    def login(self, username: str, password: str):
        """实例方法：必须使用实例调用，访问 self.driver"""
        # 必须通过实例调用：test_obj.login()
        # 可以访问：self.driver, self.base_url 等实例属性
        self.driver.find_element("id", "user").send_keys(username)
        self.driver.find_element("id", "pwd").send_keys(password)
        self.driver.find_element("id", "btn").click()

# 使用方式
test_obj = TestLogin()           # 先创建实例
test_obj.setup_driver()          # 实例调用，操作 test_obj.driver
test_obj.login("admin", "123")   # 实例调用，访问 test_obj.driver


# ========== 2. 静态方法（Static Method）==========
class ExcelHandler:
    def __init__(self, file_path: str):
        self.file_path = file_path

    @staticmethod
    def read_excel_data(file_path: str) -> List[Dict]:
        """
        静态方法：不需要 self，也不需要 cls
        纯粹的工具函数，逻辑上归类到 ExcelHandler 下
```

```python
        """
        # 不能访问：self.file_path（因为没有 self）
        # 不能访问：类变量（因为没有 cls）
        # 只能通过参数传入数据
        print(f"读取文件: {file_path}")
        return [{"case_id": 1}, {"case_id": 2}]

    @staticmethod
    def is_valid_row(row: dict) -> bool:
        """静态方法：纯逻辑判断，与类/实例状态无关"""
        return "case_id" in row and "keyword" in row

# 使用方式（两种都可以）
# 方式 A：类直接调用（推荐，表明与实例无关）
data = ExcelHandler.read_excel_data("cases.xlsx")
valid = ExcelHandler.is_valid_row({"case_id": 1})

# 方式 B：实例调用（也能跑，但语义不清）
handler = ExcelHandler("temp.xlsx")
data = handler.read_excel_data("cases.xlsx")  # 静态方法忽略 self


# ========== 3. 类方法（Class Method）==========
class WebDriverFactory:
    # 类属性：所有实例共享的默认配置
    default_options = {"headless": False, "window_size": "1920x1080"}
    instance_count = 0

    def __init__(self, browser_type: str):
        self.browser_type = browser_type  # 实例属性
        WebDriverFactory.instance_count += 1  # 修改类属性

    @classmethod
    def create_chrome_driver(cls, headless: bool = False):
        """
        类方法：第一个参数是 cls（类本身），不是 self（实例）
        可以访问类属性，但不需要创建实例
        """
        # 可以访问：cls.default_options, cls.instance_count
        # 可以调用：cls() 来创建实例（工厂模式）

        print(f"当前类配置: {cls.default_options}")
        print(f"已创建实例数: {cls.instance_count}")

        # 修改类属性（影响所有实例）
        if headless:
            cls.default_options["headless"] = True

        # 返回实例（工厂模式常用）
        return cls(browser_type="chrome")

    @classmethod
    def update_default_options(cls, **kwargs):
        """类方法：修改全局配置，影响所有后续实例"""
        cls.default_options.update(kwargs)

# 使用方式（两种都可以）
# 方式 A：类直接调用（常用作工厂方法）
```

```
driver = WebDriverFactory.create_chrome_driver(headless=True)
# 输出：当前类配置: {'headless': False...}，已创建实例数: 0

# 方式 B：实例调用（cls 自动指向 WebDriverFactory）
factory = WebDriverFactory("firefox")
factory.create_chrome_driver()  # cls 仍然是 WebDriverFactory，不是 factory

# 修改类属性（所有实例共享）
WebDriverFactory.update_default_options(headless=True, timeout=30)
```

- **实例方法**："我要用这个对象自己的东西"（`self.` 开头）
- **静态方法**："我只是个工具人，别把我当对象"（`@staticmethod`）
- **类方法**："我管的是整个类的集体事务"（`cls.` 开头）

总结：对象(个人特点) 的 工具（@classmethod） 能 管理全部（cls=类名）

5. pathlib

```
from pathlib import Path
base_dir=Path(__file__).resolve().parent.parent
file_path=base_dir / 'data' / 'test_cases.xlsx'
# 兼容跨平台
```

6. POM(page object model)

页面对象+测试逻辑(元素定位+测试逻辑)+数据；把每个页面封装成"页面对象"，元素定位集中管理；只关心"做什么"，不关心"怎么做"。

- step1：基础方法包装（基础类）库（单一动作）
- step2：元素+动作（继承基础类）页面（单一动作构成某一步骤）

  return self ：方法链式调用，调用方法不用重复写实例变量；只有在返回具体指以及状态时才不用返回slef(当前对象所在进度)

  ❌ **传统写法（繁琐）：**

  Python 复制

  ```python
  baidu = BaiduPage(driver)
  baidu.open_homepage()       # 返回 None
  baidu.search("追觅科技")     # 又得重新写对象名
  baidu.hover_to_setting()    # 继续重复...
  baidu.click_search_setting()
  ```

  ✅ **链式调用（流畅）：**

  Python 复制

  ```python
  baidu = BaiduPage(driver)
  baidu.open_homepage().search("追觅科技").hover_to_setting().click_search_setting()
  # 或者换行写（更像业务步骤）：
  baidu.open_homepage() \
        .search("追觅科技") \
        .hover_to_setting() \
        .click_search_setting()
  ```

- step3：组装动——>流程（test类）测试案列（组织步骤构成测试流程逻辑）

7. 返回值类型不确定时，如何写类型注解？

- 任何类型都可能返回：

```
from typing import Any
# 任何可能下选一
def get_config(key: str) -> Any:
    # 可能返回 str, int, list, dict...
    return json.loads(config_file[key])
```

- 已知有限范围的类型：

```
from typing import Union
# 给定范围多选一
def parse_value(value: str) -> Union[int, float, str, bool]:
    # 可能是整数、浮点、字符串或布尔
    if value.isdigit():
        return int(value)
    if value.replace('.', '').isdigit():
        return float(value)
    if value in ('True', 'False'):
        return value == 'True'
    return value
```

- 二选一

```
from typing import Optional
def find_element(locator: tuple) -> Optional[WebElement]:
    try:
        return driver.find_element(*locator)
    except NoSuchElementException:
        # return 与rasie不可以一起使用，必须存在，但没有就得抛出问题异常，pytest处理
        return None  # 明确返回 None，可用于判断某个元素是否存在，不存在(异常)则返回None
# 使用时要判空：
element = find_element((By.ID, "kw"))
if element is not None:  # 类型检查器会收窄类型为 WebElement
    element.click()
```

- 泛型，任意类型

```
from typing import TypeVar,List
T = TypeVar('T')  # 泛型，可以是任意类型

def get_first_item(items: List[T]) -> Optional[T]:
    """不管列表里是什么类型，返回同类型或 None"""
    return items[0] if items else None
# 使用：
numbers: List[int] = [1, 2, 3]
first_num = get_first_item(numbers)  # 推断为 Optional[int]

names: List[str] = ["a", "b"]
first_name = get_first_item(names)  # 推断为 Optional[str]
```

- 自定义固定值

```python
from typing import Optional,Literal
Locater = tuple[str, str]

class PageBase:
    def find_element(self, locator: Locater) -> Optional[WebElement]:
        """可能找不到，返回 Optional"""
        try:
            return self.wait.until(EC.presence_of_element_located(locator))
        except TimeoutException:
            return None

    def get_status(self) -> Literal["success", "fail", "pending"]:
        """只能是这三个字符串之一"""
        # Literal 是 Python 3.8+ 的特性，用于限定取值范围
        return "success"
```

8. 跨目录导入公共库（最佳实践）

- **小项目：**

pytest 会自动加载 `conftest.py` 中的 fixture 和导入，且它所在目录被视为根目录，重点是你所放的位置。

```
# conftest.py 支持多层级放置，遵循就近原则（类似 Python 的变量作用域）。
selenium-learning/                  # 根目录（全局作用域）
├── conftest.py                     # ← 全局 fixtures：driver、数据库连接、基础配置
├── fixtures/                 # 新建目录，存放各类 fixtures
|   ├── __init__.py           # 空文件，让 Python 识别为包
|   ├── db_fixtures.py        # 数据库相关
|   ├── web_fixtures.py       # 浏览器/UI 相关
|   └── data_fixtures.py      # 测试数据相关
├── config/
|   └── imports.py
├── day06/                          # 模块级 作用域
|   ├── conftest.py               # ← Day6 特有：BaiduPage 初始化、百度 URL 配置
|   └── test_cases/
|       ├── conftest.py           # ← 测试级 作用域：Excel 数据预处理、截图路径
|       └── TestBaiduPOM.py       # 自动继承上级所有 fixtures
├── day07/                          # 另一个模块
|   ├── conftest.py               # ← Day7 特有：API 测试的 session、token
|   └── test_api/
|       └── TestUserAPI.py
```

位置	有效范围	优先级
根目录 `conftest.py`	整个项目	最低（兜底）
模块级 `day06/conftest.py`	`day06/` 及其子目录	中（覆盖根目录）
测试级 `day06/test_cases/conftest.py`	仅该目录下测试文件	最高（最优先）

conftest.py 根目录就是比模块级更多文件目录，模块级也是管理更多文件目录但是范围比根目录小，测试级比前两者跟小，只管理测试那几个文件，以及还有文件里面的夹具范围最小。如果有重复，范围小的说了算。

被夹具装饰的方法，同名且范围小的覆盖范围大的。同级别启用。autouse=True, 不同名叠加。

```
# fixtures/db_fixtures.py
import pytest
import pymysql

@pytest.fixture(scope="session")
def db_connection():
    """数据库连接（跨测试会话共享）"""
    conn = pymysql.connect(host="localhost", user="test", password="123456",
database="test_db")
    print("数据库连接已建立")
    yield conn
    conn.close()
    print("数据库连接已关闭")

@pytest.fixture # 默认function
def db_cursor(db_connection):
    """数据库游标（每个测试函数新建）"""
    cursor = db_connection.cursor()
    yield cursor
    cursor.close()
```

```
# conftest.py（项目根目录 模块级）
import sys
from pathlib import Path

# 自动将项目根目录加入 Python 路径
ROOT_DIR = Path(__file__).parent
sys.path.insert(0, str(ROOT_DIR))

# 预导入常用模块，让测试文件直接可用（可选）
pytest_plugins = [
    "fixtures.db_fixtures",    # 对应 fixtures/db_fixtures.py
    "fixtures.web_fixtures",   # 对应 fixtures/web_fixtures.py
    "fixtures.data_fixtures",  # 对应 fixtures/data_fixtures.py
]

# 定义全局 fixture（所有测试文件自动可用）
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

@pytest.fixture(scope="session")
def global_driver():
    """全局 driver，整个测试会话只启动一次"""
    options = Options()
    driver = webdriver.Chrome(options=options)
    yield driver
    driver.quit()
```

- **大项目**

pip install -e .（最专业，模拟真实库）

```
#创建 setup.py（项目根目录）：
setup(
    name="selenium-learning",           # 包名（pip list 中显示的名字）
    version="0.1",                      # 版本号
    packages=find_packages(),           # 自动发现所有包（含 __init__.py 的文件夹）
    install_requires=[                  # 依赖（可选）
        'selenium>=4.0',
        'pytest',
        'openpyxl'
    ],
    python_requires='>=3.8',            # Python 版本要求
)
```

```
# 在项目根目录（setup.py 所在目录）
cd C:\Users\...\selenium-learning

# 安装（带 -e 表示开发模式）
pip install -e .
# 放一个指向源码目录的链接，创建"软链接"
```

```
pip uninstall selenium-learning
```

9. PageFactory

准备:

```
# conftest.py  根级
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

@pytest.fixture(scope="function")
def driver():
    """浏览器驱动 fixture - 这就是上面代码里用的 driver"""
    options = Options()
    options.add_argument("--start-maximized")
    driver = webdriver.Chrome(options=options)
    yield driver
    driver.quit()
```

```
# page_factory.py（放在 helpers/ 或 tools/ 下）
from typing import Type, TypeVar, Dict
from selenium.webdriver.remote.webdriver import WebDriver
# 使用的本地webdriver也是继承该类，只不过直接默认本地，该remote.webdriver可以远程控制

# 泛型，表示任何 Page 子类
T = TypeVar('T', bound='BasePage')
# 表示T的类型必须是BasePage或者是BasePage的子类

class PageFactory:
    """
    页面对象工厂
```

```
    1. 管理 driver 实例
    2. 缓存已创建的页面对象（避免重复实例化）
    3. 提供统一的页面切换方法
    """

    def __init__(self, driver: WebDriver):
        self.driver = driver
        self._pages: Dict[str, object] = {}  # 缓存池

    def get_page(self, page_class: Type[T]) -> T:
        """
        通用获取页面方法
        用法：baidu = factory.get_page(BaiduPage)
        """
        page_name = page_class.__name__

        # 如果已经创建过，直接返回（单例模式）
        if page_name not in self._pages:
            print(f"创建新页面实例: {page_name}")
            self._pages[page_name] = page_class(self.driver)

        return self._pages[page_name]
        # 已存在该页面类，则不在新创建，直接返回原有的页面类；实现页面类管理

    def clear_cache(self):
        """切换测试场景时清空缓存"""
        self._pages.clear()

    def remove_page(self, page_class: Type[T]):
        """移除特定页面缓存（强制重新创建）"""
        page_name = page_class.__name__
        if page_name in self._pages:
            del self._pages[page_name]

# 使用示例（在测试类中，也就是要测试的不同页面，以不同页面为单位来写测试类（页），测试流程）：
# 尽量做到，基类只给子类使用，子类（方法可同名重写）只给测试类使用。
from page_object_model.BaiduPage import BaiduPage
from page_object_model.LoginPage import LoginPage  # 假设你有这个

class TestWorkflow:
    # 测试级
    @pytest.fixture(autouse=True)
    # 自动创建夹具，不用测试方法的参数再次写夹具装饰的方法名，参数driver是conftest下的
fixture
    def setup_factory(self, driver):
        """每个测试方法自动初始化 factory"""
        # 自动调用上面创建的页面工厂类
        self.factory = PageFactory(driver)

    # 真正测试方法
    def test_full_workflow(self):
        # 获取页面（自动缓存）
        baidu = self.factory.get_page(BaiduPage)
        baidu.open_target_page("https://www.baidu.com")

        # 假设点击登录跳转到登录页
        baidu.click_login()
```

```
        # 由于点击了登录条状，所以接下来需要再次使用工厂类，管理页面。控制不同页面了，如果已经实例化则继续使用旧有的页面类实例，而不是再次创建一个新的页面类。
        # 获取登录页（factory 自动管理，driver 状态保持一致）
        login = self.factory.get_page(LoginPage)
        login.login("username", "password")

        # 回到百度页（从缓存取，不会重新创建）
        baidu = self.factory.get_page(BaiduPage)
        baidu.search("追觅")
```

10. YAML 配置化

```
page_object_model/
├── configs/
|   ├── baidu_page.yaml       # 定位符配置
|   └── login_page.yaml
├── base_page.py              # 读取 YAML 的基类
└── baidu_page.py             # 业务方法（无硬编码定位符）
```

```
#baidu_page.yaml:
百度首页:
  搜索框: {by: "id", value: "kw"}
  搜索按钮: {by: "id", value: "su"}
  设置链接: {by: "link text", value: "设置"}
```

```
# 对basePage进行方法添加
# base_page.py
import yaml
from pathlib import Path
from selenium.webdriver.common.by import By

# 可以单独提一页
class ConfigurablePage:
    """支持 YAML 配置的基类"""

    def __init__(self, yaml_file: str):
        self.locators = self._load_locators(yaml_file)

    def _load_locators(self, yaml_file: str) -> dict:
        """加载 YAML 定位符"""
        config_path = Path(__file__).parent / "configs" / yaml_file
        with open(config_path, 'r', encoding='utf-8') as f:
            config = yaml.safe_load(f)
        # 扁平化配置（把嵌套字典转成 {name: (by, value)}）
        locators = {}
        for page_name, elements in config.items():
            for elem_name, locator in elements.items():
                locators[elem_name] = (locator['by'], locator['value'])
        return locators

    def get_locator(self, name: str) -> tuple:
        """获取定位符"""
        if name not in self.locators:
            raise KeyError(f"未找到元素 '{name}' 的定位配置，请检查 YAML 文件")
        by, value = self.locators[name]
        # 字符串转 By 对象
```

```
        by_map = {
            'id': By.ID, 'name': By.NAME, 'class': By.CLASS_NAME,
            'xpath': By.XPATH, 'css': By.CSS_SELECTOR,
            'link text': By.LINK_TEXT, 'partial link text': By.PARTIAL_LINK_TEXT
        }
        return (by_map.get(by, By.ID), value)

# 使用（BaiduPage 变得超级简洁）：
# BaiduPage继承多个父类，分别使用父类的初始化
class BaiduPage(ConfigurablePage,BasePage):
    def __init__(self,yaml_file:str='element_locator.yaml',driver:webdriver):
       ConfigurablePage.__init__(yaml_file)
        BasePage.__init__(driver)
        #如果 PageBase 和 ConfigurablePage 都有 self.driver，后初始化的会覆盖前面的。

    def search(self, keyword: str):
        # 不再写死 (By.ID, "kw")，而是从 YAML 读取
        self.find("搜索框").send_keys(keyword)
        self.find("搜索按钮").click()
```

11. 自定义标记

```
# conftest.py 注册标记（避免警告）
def pytest_configure(config):
    config.addinivalue_line("markers", "slow: marks tests as slow")
    config.addinivalue_line("markers", "smoke: marks tests as smoke tests")

# 测试文件中使用：
@pytest.mark.smoke  # 冒烟测试
def test_critical_path():
    pass

@pytest.mark.slow   # 慢测试（可能跳过）
def test_heavy_load():
    pass

# 命令行只跑冒烟测试：
# pytest -v -m "smoke"
# 排除慢测试支持正则，逻辑表达式：
# pytest -v -m "not slow"
```

12. 失败重试（flaky 测试处理）

```
pip install pytest-rerunfailures
```

```
@pytest.mark.flaky(reruns=3, reruns_delay=1)  # 失败重试 3 次，间隔 1 秒
def test_unstable_feature():
    # 可能不稳定的测试（如网络抖动）
    assert random.choice([True, False])
```